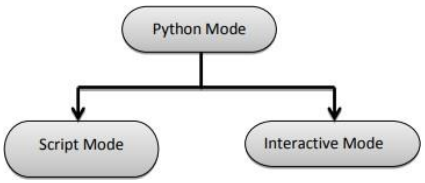
UNIT 1 INTRODUCTION TO PYTHON

Introduction to python – Features of python – Installation and execution – Variables and data types – Input and output statements – Operators and expressions – Type conversion – Python programming examples.

1.INTRODUCTION TO PYTHON

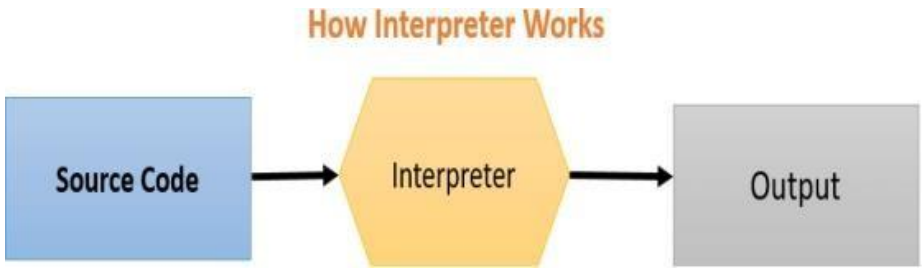
The Python programming language was conceived in the late 1980s, and its implementation was started in December 1989 by Guido van Rossum at CWI in the Netherlands as a successor to the ABC programming language. When he began implementing Python, Guido van Rossum was also reading the published scripts from "Monty Python's Flying Circus‖, a BBC comedy series from the 1970s. Van Rossum thought he needed a name that was short, unique, and slightly mysterious, so he decided to call the language Python.

Python is a cross-platform programming language, meaning, it runs on multiple platforms like Windows, Mac OS X, Linux, Unix and has even been ported to the Java and .NET virtual machines. It is free and open source.



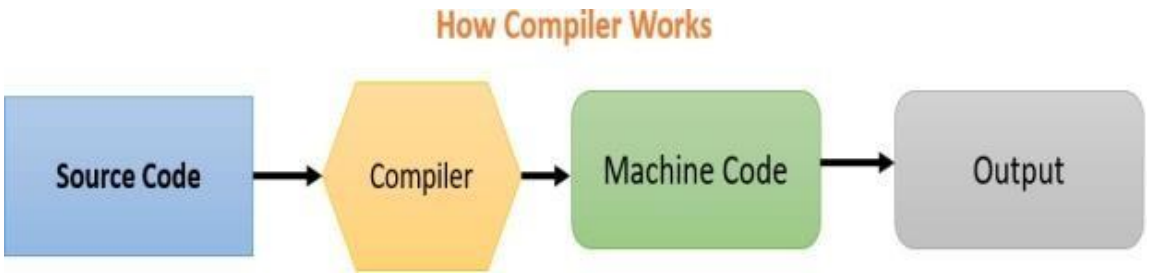
Interpreter:

It is a program that reads the high-level program (source code) and executes the program line by line. Then, finally performs computations and prints the output. Thus, the structure of interpreter is shown below.



Compiler:

A compiler is a computer program that transforms code written in a high-level programming language into the machine code. It is a program which translates the human-readable code to a language a computer processor understands (binary 1 and 0 bits). The computer processes the machine code to perform the corresponding tasks.



2.FEATURES OF PYTHON

- Python is an Open Source: It is freely downloadable, from the link "http:// python.org/".
- Python is portable: It runs on different operating systems / platforms3.
- Python has automatic memory management.
- Python is flexible with both procedural oriented and object-oriented programming.
- Python is easy to learn, read and maintain.
- Python is Extendable. You can add low-level modules to the Python is interpreted.
- These modules enable programmers to add to or customize their tools to be more efficient.
- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.
- It supports functional and structured programming methods as well as OOPS.

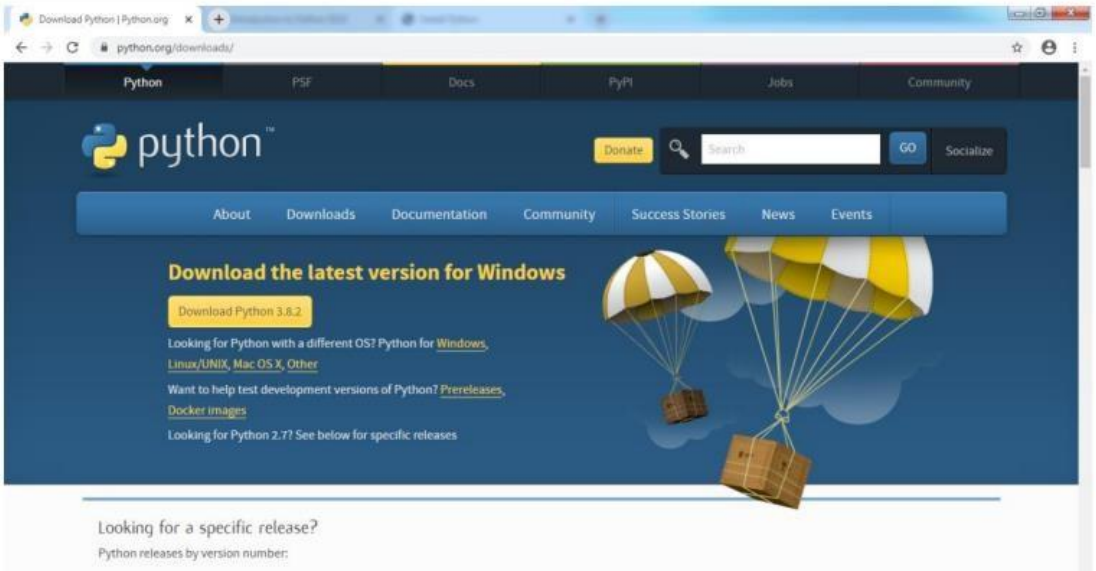
- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.
- It supports functional and structured programming methods as well as OOPS.

- It is very flexible with the console program, Graphical User Interface (GUI) applications, Web related programs etc.

- **Easy to code:** Python is a high-level programming language as it is easy to comprehend as compared to other language like c, c#, Java script, Java and so forth, one can effortlessly learn and code in python barely in hours. Additionally, it is also a developer-friendly language.
- **Platform Independent:** Python programs can be developed and executed on numerous operating system framework. Python can be used on Linux, Windows, Macintosh, Solaris and some others.
- **Object-Oriented Language**: Python bolsters object-oriented language and concepts of classes, objects encapsulation etc.
- **Free and Open Source:** Python language is freely available at the official website. Since, it is open-source, available to the public. So, one can download it, use it as well as share it.
- **GUI Programming Support:** Graphical Users interfaces can be made using a module such as PyQt5, PyQt4, wxPython or Tk in python.
- **High-level Language:** Python is a high-level language. When one develops programs in python, he/she didn't need to memorize the system architecture or to manage the memory.
- **Portable language:** Python is a portable language, for instance, on the off chance that the code written in python for windows can also run on different other platforms such as Linux, Unix and Mac etc.
- **Integrated and Interpreted Language:** Python is an Interpreted Language, since python code is executed line by line at a time. Python is additionally an integrated language since one can without much of a stretch, can integrate python with another language like C, C++ and so on.

3.INSTALLATION AND EXECUTION

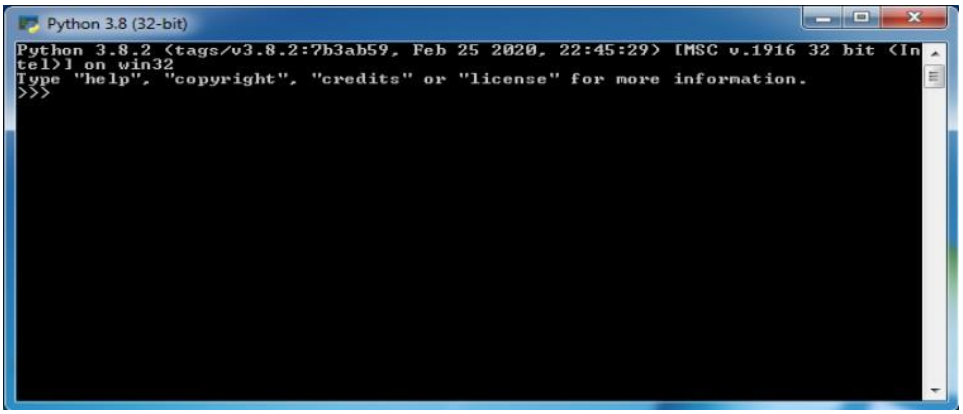
Python is a freeware that can be installed on your workstation or laptop. The current version of is Python 3.8.2. (Release date: Feb 24, 2020). Python can be downloaded from the https://www.python.org/downloads/Website



After installation, shortcut for the python software will be available on the start menu. Select Start menu -> All Programs -> Python 3.8 -> IDLE Python 3.8 to invoke python IDE (Integrated Development Environment).



IDLE (Integrated Development and Learning Environment) is an IDE for Python. Python programs can also be executed in Python command line



PROGRAM:

- A program performs a task in the computer. But, in order to be executed, a program must be written in the machine language of the processor of a computer.
- Unfortunately, it is extremely difficult for humans to read or write a machine language program.
- This is because a machine language is entirely made up of sequences of bits.
- However, high level languages are close to natural languages like English and only use familiar mathematical characters, operators and expressions.
- Hence, people prefer to write programs in high level languages like C, C++, Java, or Python.
- A high-level program is translated into machine language by translators like compiler or interpreter.

EXECUTION IN PYTHON

Execution in Python means running a Python program so that the computer processes the instructions and produces the output.

1. Types of Execution in Python

1.1 Interactive Mode

- In interactive mode, Python executes each statement immediately after you enter it.
- You can use the Python to test small pieces of code.

Example:

```
>>> print("Hello World")
```

Output:

```
Hello World
```

Advantages:

- Easy to test small programs.
- Gives immediate results.
- Useful for learning Python commands.

1.2 Script Mode

In script mode, you write Python statements in a file with the .py extension and execute the entire file.

Example1:

```
print("Welcome to Python")
```

Output:

```
Welcome to Python
```

Example2:

```
a = 10
b = 20
print("Sum =", a + b)
```

Output:

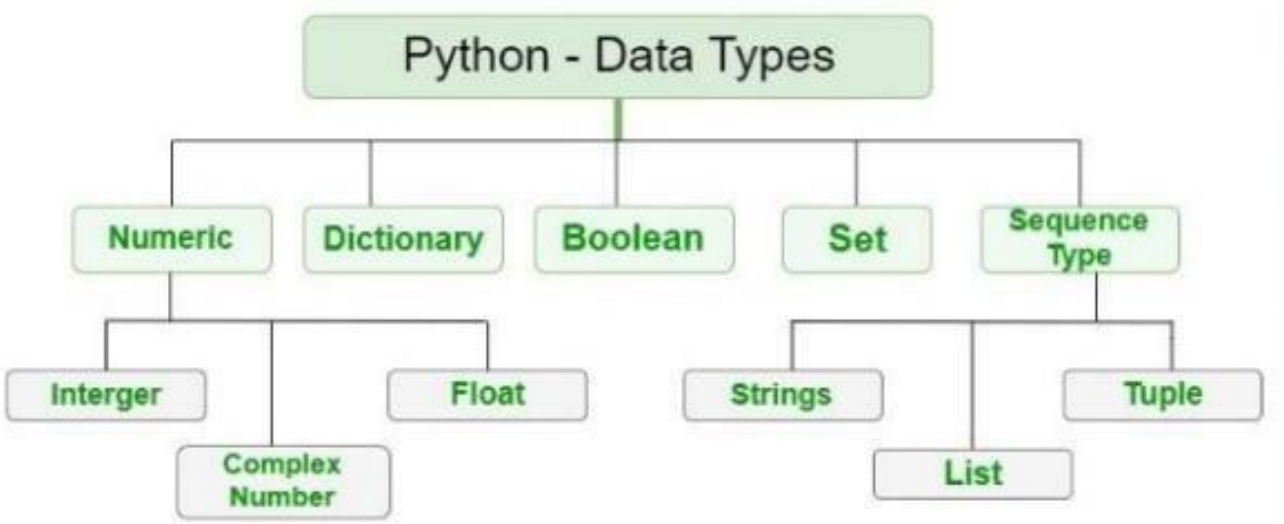
```
Sum = 30
```

2. Ways to Execute Python Programs

- **IDLE:** Write and run Python programs.
- **VISUAL STUDIO CODE:** Run Python files using the Python extension.
- **COMMAND PROMPT**: Execute files using python filename.py.
- **JUPYTER NOTEBOOK:** Run code one cell at a time.
- **ONLINE PYTHON COMPILER:** Write and execute programs in a web browser.

4.DATA TYPES IN PYTHON

A Data type indicates which type of value a variable has in a program. However, a python variable can store data of any data type but it is necessary to identify the different types of data they contain to avoid errors during execution of program. The most common data types used in python are str(string), int(integer) and float (floating-point).



Data Types	
Numeric Types:	**int, float, complex**
Boolean Type:	**bool**
Text Type:	**str**
Sequence Types:	**list, tuple, range**
Mapping Type:	**dict**
Set Types:	**set, frozen set**
Binary Types:	**bytes, byte array, memory view**

4.1NUMERIC TYPES

INTEGERS AND FLOAT

- A data type is a category for values, and every value belongs to exactly one datatype.

- All data values in Python are encapsulated in relevant object classes. The most common data types in Python are listed below.

Example

Data Type	Examples
Integers	-2, -1, 0, 1, 2, 3, 4, 5
Floating-point numbers	-1.25, -1.0, --0.5, 0.0, 0.5, 1.0, 1.25

4.2 COMPLEX

Complex numbers are written with a "j" as the imaginary part:

Example

```
>>>x = 3+5j
>>>y = 5j
>>>z= -5j
>>>print(type(x))
>>>print(type(y))
>>>print(type(z))
```

Output

```
<class,,complex‟>
<class,,complex‟>
<class,,complex‟>
```

4.3BOOLEAN TYPE

- Boolean is a data type named after George Boole (1815-1864).
- A Boolean variable can take only two values, True or False.
- The main use of this type is in logical expressions.

Example

```
a = True
b = 30>45       # b gets the value False
```

Boolean expressions are often used in conjunction with the if statement

Boolean operators

Boolean operations are performed using the and, or, & not keywords in Python

Example	Result
True and False	False
False or True	True
(30> 45) or(27<30)	True
not True	False
not (3> 4)	True

4.4VARIABLES

- A variable is a name that refers to a value.
- Variable reserved memory locations to store values.
- This means that when you create a variable you reserve some space in memory.
- Based on the data type of a variable, the interpreter allocates memory and decides what can be stored in the reserved memory.

Assignment Statements

An assignment statement creates a new variable and gives it a value.

```
>>>message = 'Introducing Python Variable'
>>>num = 15
>>>radius = 5.4
```

- This example makes three assignments.
- The first assigns a string to a new variable named message.
- The second gives the integer 15 to num.
- The third assigns floating point value 5.4 to variable radius.

Variable Names

- Programmers generally choose names for their variables that are meaningful the rules
- Variables names must start with a letter or an underscore, such as.
    - o _mark
    - o mark_
- The remainder of your variable name may consist of letters, numbers and underscores.
    - subject1
    - my2ndsubject
    - un_der_scores

Names are case sensitive

- case_sensitive, CASE_SENSITIVE, and Case_Sensitive are each a different variable. Can be any (reasonable) length.

Reserved Words

- There are some reserved (Key Words) words which you cannot use as a variable name because Python uses them for other things.
- The interpreter uses keywords to recognize the structure of the program, and they cannot be used as variable names.

5.INPUT AND OUTPUT STATEMENTS IN PYTHON

5.1 Introduction

- In Python, input and output statements are used to communicate between the user and the program.
- Input: Used to get data from the user.
- Output: Used to display information or results on the screen.

Python mainly uses two functions:

1. input() – to receive input from the user.
2. print() – to display output on the screen.

5.2 Output Statement – print()

- The print() function is used to display messages, values, and results.

Syntax

```
print("message")
```

Example1: Display a message

```
print("Welcome to Python Programming")
print("Hello World")
```

Output

```
Welcome to Python Programming
Hello World
```

Example2: Display variables

```
name = "Ravi"
age = 20
print(name)
print(age)
```

Output

```
Ravi
20
```

Example3: Display calculations

```
a = 10
b = 20
print(a + b)
```

Output

```
30
```

5.3Input Statement – input()

- The input() function is used to get information from the user through the keyboard.

Syntax

variable = input("Enter a value: ")

Example1: Read a name

```
name = input("Enter your name: ")
print("Welcome", name)
```

Output

```
Enter your name: Arun
Welcome Arun
```

Example2: Read a number

```
number = input("Enter a number: ")
print("You entered:", number)
```

Output

```
Enter a number: 25
You entered: 25

Enter a number: 25
You entered: 25
```

Example3: Addition of two numbers

```
a = int(input("Enter the first number: "))
b = int(input("Enter the second number: "))
sum = a + b
print("Sum =", sum)
```

Output

```
Enter the first number: 10
Enter the second number: 20
Sum = 30
```

Example4: Calculate the average of two numbers

```
a = float(input("Enter the first mark: "))
b = float(input("Enter the second mark: "))
average = (a + b) / 2
print("Average =", average)
```

Output

Enter the first mark: 80
Enter the second mark: 90
Average = 85.0

5.4 Multiple Values in Input and Output

- Python allows users to enter multiple values in one line using split().

```
a, b = input("Enter two numbers: ").split()
print("First number:", a)
print("Second number:", b)
```

Output

Enter two numbers: 10 20
First number: 10
Second number: 20

Here, both values are strings. To add them as numbers, use:

```
a, b = map(int, input("Enter two numbers: ").split())
print("Sum =", a + b)
```

Output

Enter two numbers: 10 20
Sum = 30

5.5 Simple Program Using Input and Output

Program: Calculate the area of a rectangle

```
length = float(input("Enter the length: "))
breadth = float(input("Enter the breadth: "))
area = length * breadth
print("Area of rectangle =", area)
```

Output

Enter the length: 5
Enter the breadth: 4
Area of rectangle = 20.0

6.OPERATORS AND EXPRESSIONS IN PYTHON

6.1Introduction

- In Python, operators are special symbols or keywords used to perform operations on variables and values.
- An expression is a combination of values, variables, and operators that produces a result.

Example:

```
a = 10
b = 5
result = a + b
print(result)
```

Output

15

6.2Types of Operators in Python

Python supports the following major types of operators.

1. Arithmetic Operators
Perform mathematical calculations.
+, -, *, /, %, //, **

2. Comparison Operators
Compare two values and return True or False.
==, !=, >, <, >=, <=

3. Logical Operators
Combine or reverse conditions.
and, or, not

4. Assignment Operators
Assign or update variable values.
=, +=, -=, *=, /=, %=

5. Bitwise Operators
Perform operations on the binary bits of integers.
&, |, ^, ~, <<, >>

6. Membership Operators
Check whether a value exists in a sequence or collection.
in, not in
7. Identity Operators
Check whether two references point to the same object.
is, is not

6.2.1Arithmetic Operators

- Arithmetic operators are used to perform mathematical calculations.

Operator	Meaning	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2.0
%	Modulus (remainder)	10 % 3	1
//	Floor division	10 // 3	3
**	Exponentiation	2 ** 3	8

Program

```
a = 10
b = 3
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Remainder:", a % b)
print("Floor Division:", a // b)
print("Power:", a ** b)
```

Output

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Remainder: 1
Floor Division: 3
Power: 1000
```

6.2.2Comparison Operators

- Comparison operators compare two values and return a Boolean result (True or False).

Operator
Meaning
Example
Result
1.==
Equal to

10 == 10
True
2.!=
Not equal to
10 != 5
True
3.>
Greater than
10 > 5
True
4.<
Less than
10 < 5
False
5.>=
Greater than or equal to
10 >= 10
True
6.<=
Less than or equal to
5 <= 10
True

Example1:

```
a = 20
b = 10
print(a == b)
print(a != b)
print(a > b)
print(a < b)
```

Output

```
False
True
True
False
```

Example2:

```
age = 20
print(age >= 18 and age <= 60)
print(age < 18 or age > 60)
print(not (age < 18))
```

Output

```
True
False
True
```

6.2.3Assignment Operators

- Assignment operators assign values to variables or update their existing values.

Example

```
x = 10
x += 5
print(x)
x *= 2
print(x)
```

Output

```
15
30
```

6.2.4Membership Operators

- Membership operators check whether an element exists in a sequence, such as a string, list, or tuple.

Example

```
fruits = ["apple", "banana", "orange"]
print("apple" in fruits)
print("grapes" not in fruits)
```

Output

```
True
True
```

6.2.5Identity Operators

- Identity operators check whether two variables refer to the same object in memory.
- is — returns True if both references point to the same object.
- is not — returns True if they point to different objects.

Example

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a is b)
print(a is c)
print(a == c)
```

Output

```
True
False
True
```

6.2.6Bitwise Operators

- Bitwise operators work on the binary representation of integers.

- | Operator | Meaning | |---|---| | & | Bitwise AND | | | | Bitwise OR | | ^ | Bitwise XOR | | ~ | Bitwise NOT | | << | Left shift | | >> | Right shift |

Example

```
a = 5
b = 3
print(a & b)
print(a | b)
print(a ^ b)
print(a << 1)
print(a >> 1)
```

Output

```
1
7
6
10
2

1
7
6
10
2
```

EXPRESSIONS IN PYTHON

- An expression is a combination of constants, variables, operators, and sometimes function calls that Python evaluates to produce a value.

```
10 + 20
a * b
x > 10
(5 + 3) * 2
```

Types of expressions

- Arithmetic expression: a + b * 2
- Comparison expression: a > b
- Logical expression: a > 5 and b < 10
- String expression: "Hello" + " Python"
- Conditional expression: result = "Pass" if mark >= 40 else "Fai

Program

```
a = 10
b = 5
sum_result = a + b
difference = a - b
product = a * b
average = (a + b) / 2
print("Sum:", sum_result)
print("Difference:", difference)
print("Product:", product)
print("Average:", average)
```

Output

```
Sum: 15
Difference: 5
Product: 50
Average: 7.5
```

7.TYPE CONVERSION IN PYTHON

7.1 Introduction

- Type conversion in Python is the process of changing a value from one data type to another.
- It is useful when performing calculations, accepting input from users, and processing different types of data.
- For example, converting an integer into a float or a string into an integer is called type conversion.
- Python supports two types of type conversion

1. Implicit Type Conversion
2. Explicit Type Conversion

7.2 Types of Type Conversion

A. Implicit Type Conversion

Implicit type conversion occurs when Python automatically converts one data type into another without requiring the programmer to specify the conversion.

Example

```
a = 10
b = 2.5
c = a + b
print(c)
print(type(c))
```

Output

12.5
<class 'float'>

Explanation

- a is an integer (int).
- b is a floating-point number (float).
- Python automatically converts a into a float during the calculation.
- The result is 12.5, which is a float.

B. Explicit Type Conversion

- Explicit type conversion occurs when the programmer manually converts a value from one data type to another using built-in functions.
- It is also called type casting.

Common type conversion functions are

1.int()
Converts to an integer
int(5.8)
5

2.float()
Converts to a float
float(10)
10.0

3.str()
Converts to a string
str(100)
"100"

4.bool()
Converts to a Boolean value
bool(1)
True

5.list()
Converts an iterable to a list
list("abc")
['a', 'b', 'c']

6.tuple()
Converts an iterable to a tuple
tuple([1, 2])
(1, 2)

7.set()
Converts an iterable to a set
set([1, 2, 2])
{1, 2}

Examples of Explicit Type Conversion

Example1: Integer Conversion Using int()

```
a = 15.9
b = int(a)
print(b)
print(type(b))
```

Output

```
15
<class 'int'>
```

Example2: Float Conversion Using float()

```
a = 25
b = float(a)
print(b)
print(type(b))
```

Output

```
25.0
<class 'float'>
```

Example3: String Conversion Using str()

```
age = 20
result = str(age)
print(result)
print(type(result))
```

Output

```
20
<class 'str'>
```

Explanation: The integer 20 is converted into the string "20".

Example 4: Boolean Conversion Using bool()

```
print(bool(1))
print(bool(0))
print(bool(""))
```

```
print(bool("Python"))
```

Output

```
True
False
False
True
```

8.PYTHON PROGRAMMING EXAMPLES FOR BEGINNERS

- Python is a simple, high-level programming language used for application development, data science, artificial intelligence, machine learning, and automation.

Program

```
print("Hello, World!")
```

Output

```
Hello, World!
```

Program

```
a = 10
b = 20
sum = a + b
print("Sum =", sum)
```

Output

```
Sum = 30
```

Program

Get Input from the User

```
name = input("Enter your name: ")
print("Welcome", name)
```

Output

```
Enter your name: Ravi
Welcome Ravi
```